THEORY

Preprocessor

The C++ preprocessor is a program that processes source code before the compiler sees it. It handles directives that start with a `#` symbol. The most common directive is `#include`, which effectively copies the contents of another file (like a header file) into the current file. This is essential for modular programming and using libraries. Another key directive is `#define`, which creates macros—fragments of code or constants that are expanded inline. For example, `#define PI 3.14`replaces every instance of `PI` with `3.14`. However, macros are text-based and do not respect C++ scope or type rules, which is why `constexpr` or `const` variables are preferred for constants. The preprocessor also manages conditional compilation using `#ifdef`, `#ifndef`, `#else`, and `#endif`. This is critical for "include guards," which prevent a header file from being included multiple times in a single translation unit, causing redefinition errors.

C++

```
// Example of Include Guard

#ifndef MY_HEADER_H

#define MY_HEADER_H

// Declarations here...

#endif
```

Basic Types

C++ defines several fundamental data types to represent arithmetic values. The integer types include `char` (1 byte, typically for characters), `short`, `int`, and `long`. These can be `signed` (positive and negative) or `unsigned` (non-negative only). For instance, an `unsigned int` uses all its bits for magnitude, effectively doubling the positive range compared to a signed `int`.

Floating-point types like `float` (single precision) and `double` (double precision) handle real numbers. There is also `bool` for boolean logic (`true` or `false`). The size of these types (except `char`) is platform-dependent, but `sizeof(type)` returns the size in bytes. C++ enforces strict typing, meaning implicit conversions (like `float` to `int`) can occur but may lead to data loss.

C++

```
int a = 10;

float b = 5.5f;

bool active = true;

unsigned char c = 255; // Max value for 8-bit unsigned
```

Flow Control

Flow control statements dictate the execution path of a program. Conditional branching is handled by `if`, `else if`, and `else` statements, which execute blocks based on boolean expressions. The `switch` statement allows branching to one of many `case` labels based on an integral value. For repetition, C++ provides `while` loops (pre-condition check), `do-while`loops (post-condition check, guaranteed to run at least once), and `for` loops (initialization, condition, and increment). The `for` loop is highly versatile and is the standard for iterating over sequences. Jump statements include `break` (exit a loop/switch), `continue` (skip to the next iteration), and `return` (exit a function).

C++

```
// Example: Range-based for loop (C++11)

int arr[] = {1, 2, 3};

for (int val : arr) {

    if (val == 2) continue; // Skip 2
```

```
    // Process val
}
```

Scopes and Visibility

Scope determines the lifetime and visibility of variables. A variable declared inside a block (`{}`) has **local scope** and is destroyed automatically when execution leaves that block. Variables declared outside all functions have **global scope** and exist for the program's entire run. **Namespaces** (e.g., `namespace std`) are used to group identifiers and prevent naming collisions between libraries. Visibility is distinct from scope; a variable can be in scope (alive) but not visible if it is shadowed by another variable with the same name in an inner block. In classes, visibility modifiers (`public`, `protected`, `private`) control access to members from outside the class or derived classes.

C++

```
int x = 10; // Global x

void func() {

    int x = 5; // Local x shadows global x

    {

        int x = 1; // Inner x shadows local x

    }

}
```

Pointers

A pointer is a variable that stores the memory address of another variable. You declare a pointer with `*` (e.g., `int* p`). The address-of operator `&` retrieves a variable's address (`p = &a`), and the dereference operator `*`

accesses the value at that address (`*p = 5` modifies `a`). Pointers are powerful for manual memory management and iterating over arrays (pointer arithmetic). A special value, `nullptr` (in C++11 and later), represents a pointer that points to nothing, replacing the older `NULL` macro. Misusing pointers, such as dereferencing `nullptr` or accessing memory after it has been freed (dangling pointer), leads to Segmentation Faults.

C++

```
int val = 42;

int* ptr = &val;

*ptr = 100; // val is now 100

ptr++;      // Moves to the next integer address (dangerous if not array)
```

Arrays

An array is a fixed-size sequence of elements of the same type stored contiguously in memory. In C++, the name of an array often decays into a pointer to its first element. Arrays do not know their own size, so when passing an array to a function, you must typically pass its size as a separate argument. Accessing an index outside the declared bounds (e.g., `arr[5]` for an array of size 5) is undefined behavior and a common source of bugs.

C++

```
int arr[3] = {10, 20, 30};

int* p = arr; // Equivalent to &arr[0]

// p[1] accesses arr[1], which is 20
```

References

A reference is an alias for an existing variable, declared using `&` (e.g., `int& ref = var`). Unlike pointers, references must be initialized upon declaration

and cannot be reseated to refer to a different variable later. They act like constant pointers that are automatically dereferenced. References are essential for passing large objects to functions ("pass by reference") to avoid the performance cost of copying, while still allowing the function to modify the original object (unless passed as `const` reference).

C++

```
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}
```

Stack and Heap

Memory is divided into the Stack and the Heap. The **Stack** is used for static memory allocation; it stores local variables and function call frames. Allocation is automatic and fast, but the size is limited and fixed. The **Heap** is used for dynamic memory allocation, managed manually via `new` (allocate) and `delete` (free). Heap memory persists until explicitly freed. Forgetting to `delete` memory causes "Memory Leaks," while deleting memory twice causes a "Double Free" crash.

C++

```
void func() {
    int stackVar = 10;          // Lives on Stack
    int* heapVar = new int(20);  // Lives on Heap
    delete heapVar;             // Must be freed manually
}
```

Structs with Methods

In C++, `struct` and `class` are functionally almost identical. The only technical difference is default visibility: members of a `struct` are `public` by default, whereas members of a `class` are `private`. Like classes, structs can have member variables, member functions (methods), constructors, and destructors. They are often used for data structures where all members are intended to be accessible, but they fully support Object-Oriented principles.

C++

```
struct Vec2 {

    float x, y;

    float norm() { return sqrt(x*x + y*y); } // Method inside struct

};
```

Constructors and Destructors

A **Constructor** is a special method called automatically when an object is instantiated to initialize its state. It has the same name as the class and no return type. A "Copy Constructor" initializes a new object as a copy of an existing one; if not defined, the compiler provides a default shallow copy. A **Destructor** (`~ClassName`) is called when an object goes out of scope or is deleted. It is responsible for cleanup, such as releasing heap memory (`delete` pointers) or closing file handles.

C++

```
class Data {

    int* buf;

public:

    Data() { buf = new int[10]; } // Constructor

    ~Data() { delete[] buf; }     // Destructor
```

```
};
```

Declaration and Definition

A **Declaration** introduces a name (variable, function, class) to the compiler and specifies its type, but does not necessarily allocate memory or provide the body. For example, `extern int x;` or `void f();` are declarations. A **Definition** provides the actual implementation or storage. `int x = 10;` and `void f() { ... }` are definitions. The "One Definition Rule" states that a specific entity can be declared many times but defined only once. Header files (`.h`) typically contain declarations (interfaces), while source files (`.cpp`) contain definitions (implementations).

Operator Overloading

Operator overloading allows user-defined types (classes/structs) to define custom behaviors for standard C++ operators (`+`, `-`, `*`, `<<`, `[]`, etc.). This makes custom types easier to use and more intuitive. For example, you can overload the `+` operator for a `Matrix` class so that `C = A + B` performs matrix addition. This is done by defining a special function `operator+`.

C++

```
struct Complex {
    float r, i;
    Complex operator+(const Complex& other) {
        return {r + other.r, i + other.i};
    }
};
```

Inheritance (Late Binding, Dynamic Cast, Multiple Inheritance)

Inheritance allows a derived class to inherit members from a base class. **Late Binding** (Polymorphism) allows a program to decide at runtime which function to call based on the object's actual type, not just its pointer type. This requires declaring the function `virtual` in the base class. **Dynamic Cast** (`dynamic_cast<Type*>(ptr)`) is used to safely convert a base pointer to a derived pointer at runtime; it returns `nullptr` if the object is not actually of the target derived type. **Multiple Inheritance** allows a class to inherit from more than one base class, though this can create ambiguity (the "Diamond Problem") which is solved using `virtual` inheritance.

C++

```
class Base { virtual void foo() {} };

class Derived : public Base {};

Base* b = new Derived();

Derived* d = dynamic_cast<Derived*>(b); // Valid cast
```

ROS Introduction

ROS (Robot Operating System) is a middleware for robotics. It organizes software into **Nodes**, which are independent processes performing specific tasks (e.g., reading a laser, controlling motors). Nodes communicate via **Topics** using a Publish/Subscribe model (asynchronous) or **Services** using a Request/Reply model (synchronous). A central **Master** node facilitates connection setup. ROS provides tools for building (Catkin/CMake), debugging, and visualization (RViz).

Metaprogramming (Templates, Function Objects, Using, Constexpr, STL)

Templates enable generic programming, allowing functions and classes to operate with generic types (e.g., `std::vector<int>` vs `std::vector<float>`). The compiler generates the specific code for each type used. **Function Objects** (Functors) are objects that can be called like functions by overloading `operator()`. The `using` keyword creates type aliases (modern replacement for `typedef`). `constexpr` tells the compiler to evaluate an expression at compile-time. The **STL (Standard Template**

Library) provides optimized implementation of **Containers** (`vector`, `list`, `map`), **Algorithms** (`sort`, `find`), **Streams** (I/O), and **Smart Pointers** (`std::shared_ptr`, `std::unique_ptr`) which automate memory management.

Eigen

Eigen is a powerful C++ template library for linear algebra (matrices, vectors, decompositions). It is the standard for math in robotics. It uses expression templates to optimize operations, avoiding temporary objects during complex math expressions like `D = A * B + C`. Key types are `Eigen::MatrixXd` (dynamic size double matrix) and `Eigen::Vector3f`(fixed size float vector of length 3).

C++

```
#include <Eigen/Dense>

Eigen::Vector2f v(1.0, 2.0);

Eigen::Matrix2f m;

m << 1, 0, 0, 1; // Identity

Eigen::Vector2f result = m * v;
```

Distance Map

A Distance Map (or Distance Transform) is a grid where every cell contains the distance to the nearest obstacle. Cells occupied by obstacles have a value of 0. As you move away from an obstacle, the value increases. This is usually computed from a binary occupancy grid using algorithms like the Chamfer distance or Breadth-First Search (BFS). It is computationally efficient for robots to query "how close am I to a wall?" in constant time O(1) by simply looking up their grid coordinates.

Navigation using ROS

ROS Navigation involves moving a robot from A to B while avoiding obstacles. The standard stack uses `move_base`. It takes **Odometry** (where the robot

thinks it moved), **Laser Scans** (what the robot sees), and a **Map**. It outputs velocity commands (`cmd_vel`). It consists of a **Global Planner** (calculates a path across the map, e.g., A*) and a **Local Planner**(calculates immediate motor commands to follow the path and avoid dynamic obstacles, e.g., Dynamic Window Approach).

Localizing on a Distance Map

Localization estimates the robot's pose (x,y,θ) in a map. A distance map allows for a "Likelihood Field Model" used in particle filters (like AMCL). For a proposed robot pose, the algorithm projects the laser scan endpoints onto the map. If the endpoints fall on cells with low distance map values (close to obstacles), the pose is likely correct. This approach is smoother and more robust than binary map matching because small errors in position don't result in a zero probability match, but rather a "lower" probability based on the distance.

Planning on a Distance Map

Path planning algorithms like Dijkstra or A* search a graph (the grid map) for the shortest path. Using a Distance Map improves this by adding a "cost" to the search. Instead of just "free" or "occupied," cells near obstacles can have a high traversal cost. This forces the planner to find a path that naturally stays away from walls (maximizing clearance), leading to safer trajectories. The distance map effectively acts as a potential field where obstacles are peaks to be avoided.

What is an Isometry (`Eigen::Isometry2f` / `3f`)?

In robotics and the Eigen library, an **Isometry** represents a **rigid body transformation**. It is a mathematical way to describe how to move points from one coordinate frame to another while preserving distances and angles between points.

Visually, think of it as moving a robot or an object without stretching, shrinking, or warping it. You are only allowed to **Rotate** it and **Translate** (move) it.

1. Mathematical Representation

An Isometry is represented by a **Homogeneous Transformation Matrix** (usually 3×3 for 2D, or 4×4 for 3D).

For a 2D robot (which is the most common case in your exam snippets), an `Isometry2f` corresponds to a 3×3 matrix structure:

$$T=\begin{bmatrix} cos\theta & -sin\theta & t_x \\ sin\theta & cos\theta & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

- **Rotation Part (2×2 top-left):** Describes the orientation (θ).
- **Translation Part (2×1 top-right):** Describes the position (tx,ty).
- **Bottom Row:** Always [0,0,1] to make matrix multiplication work with homogeneous coordinates.

2. How it is used in the Code

You will see `Isometry2f` (for 2D floats) or `Isometry3f` (for 3D floats) used constantly for **Pose** representation.

A. Constructing an Isometry

You cannot just fill the matrix manually efficiently; usually, you build it by setting the linear (rotation) and translation parts.

Example from your course snippets:

C++
```
Eigen::Isometry2f t = Eigen::Isometry2f::Identity(); // Start with "no movement"
t.translation() << 10.0, 5.0;  // Set position to x=10, y=5
t.linear() = Eigen::Rotation2Df(M_PI).toRotationMatrix(); // Set rotation to 180 deg
```

B. Transforming Points

The `*` operator is overloaded to apply the transformation to a vector. This moves a point from the "local" frame to the "global" frame defined by the isometry.

C++
```
Eigen::Vector2f point_local(1.0, 0.0); // Point 1 meter in front of robot
Eigen::Vector2f point_global = t * point_local; // Where is it in the world?
```

C. Chaining Transformations

Isometries can be multiplied together to represent a chain of coordinate frames (e.g., World → Robot → Laser → Object).

C++
```
// T_world_robot * T_robot_laser = T_world_laser
Eigen::Isometry2f global_laser_pose = robot_pose * laser_offset;
```

D. Inverting (Inverse Transformation)

To go backwards (e.g., you know a point in the World, and you want to know where it is relative to the Robot), you use `.inverse()`.

C++
```
// Where is the global point (10, 5) seen from the robot?
Eigen::Vector2f point_in_robot_frame = robot_pose.inverse() * point_world;
```

3. Key Exam Concepts to Remember

1. **Multiplication Order Matters:** A×B=B×A.
   - `robot_pose * laser_offset` is correct.
   - `laser_offset * robot_pose` is **WRONG** (mathematically valid but physically meaningless in this context).
2. **Internal Storage:** Even though it behaves like a matrix, Eigen stores Isometries efficiently. However, in the exam, you often treat it conceptually as the matrix shown above.
3. **Rotation2D vs Matrix:** `Eigen::Rotation2Df(alpha)` creates a rotation object. To put it into an Isometry, you usually call `.toRotationMatrix()`.
4. **Inverse is Cheap:** Because it is a rigid body transformation, Eigen knows a shortcut to calculate the inverse without doing a full expensive matrix inversion (it simply transposes the rotation and negates the translation vector).

1. `/odom` (Odometry)

- **What it is:** A topic that publishes the robot's estimated position and velocity based on its internal sensors (usually wheel encoders or IMU).
- **Message Type:** `nav_msgs/Odometry`.
- **Physical Meaning (Dead Reckoning):**
  - It calculates position by integrating velocity over time (xnew=xold+v⋅dt).
  - **Crucial for Exam:** Odometry is **continuous** and **smooth** (it doesn't jump), but it **drifts** over time. If the robot travels 100 meters, the `/odom` might say 98m or 102m due to wheel slippage.

C++ Structure:
C++
```
struct Odometry {
   Header header;        // Timestamp + Frame ID (usually "odom")
   string child_frame_id; // Usually "base_link"
   PoseWithCovariance pose; // Position (x,y,z) + Orientation (Quaternion)
   TwistWithCovariance twist; // Linear (vx, vy) and Angular (wz) velocities
};
```

-

- **Usage in Code:** You subscribe to this to implement controllers (PID) or to predict where the robot moved before correcting it with a map.

---

2. `/scan` (Laser Scan)

- **What it is:** A topic that provides data from a 2D LiDAR (Light Detection and Ranging) sensor.
- **Message Type:** `sensor_msgs/LaserScan`.
- **Physical Meaning:**
  - It represents a planar slice of the world.
  - It provides a list of **Ranges** (distances) measured at specific angular increments.
- **Key Math for Exam:** You often need to convert these Polar coordinates (r,θ) to Cartesian coordinates (x,y) in the robot frame.
  - $\theta_i = \text{angle_min} + i \times \text{angle_increment}$
  - $x_i = r_i \cdot \cos(\theta_i)$
  - $y_i = r_i \cdot \sin(\theta_i)$

C++ Structure:
C++
```
struct LaserScan {
   float angle_min;       // Start angle (e.g. -90 deg)
   float angle_max;       // End angle (e.g. +90 deg)
   float angle_increment; // Step between beams
   float range_min, range_max; // Valid distance limits
   vector<float> ranges;  // The actual distance measurements
};
```

What is `tf`? (The Concept)

In robotics, a robot is made of many moving parts, and the robot itself moves within the world. Each part has its own **Coordinate Frame**.

- **World/Map Frame:** The fixed origin of the room (0,0,0).
- **Base Frame:** The center of the robot.
- **Laser Frame:** The physical location where the laser scanner is mounted (e.g., 20cm forward, 10cm up from the center).

The Problem: The laser reports: "I see an obstacle 1 meter in front of *me*." **The Question:** Where is that obstacle in the *Map*?

To answer this, you need to calculate a chain of multiplications:

$$P_{map} = T_{map \to odom} \times T_{odom \to base} \times T_{base \to laser} \times P_{laser}$$

`tf` **(Transform Library)** is the ROS system that handles this background math for you. It tracks where every frame is relative to every other frame over time.

What is `tf::TransformListener`?

You typically **do not** subscribe to the `/tf` topic directly because the math to traverse the tree (finding the path from Frame A to Frame B) and handle time synchronization is complex.

Instead, you use a helper class: `tf::TransformListener`.

1. The Buffer (The "Time Machine")

When you instantiate a `tf::TransformListener` object in your code, it automatically subscribes to `/tf` and starts recording all coordinate transformations into a local **cache (buffer)** (usually defaulting to the last 10 seconds).

This allows you to ask questions like:

- "Where was the laser frame relative to the map **right now**?"
- "Where was the laser frame relative to the map **500ms ago**?"

2. The `lookupTransform` Method

This is the most critical method for the exam. It asks the listener to calculate the relationship between two frames.

Syntax:

C++

```
listener.lookupTransform(target_frame, source_frame, time, transform_storage);
```

- `target_frame`: The frame you want the coordinates *in* (e.g., "map").
- `source_frame`: The frame your data is currently *in* (e.g., "laser_link").
- `time`: The specific moment you want the transform for.
  - Use `ros::Time(0)` to get the **most recent** available transform.
  - Use `ros::Time::now()` to get the transform at the exact current microsecond (this often fails because data takes a few milliseconds to arrive/publish, so `Time(0)` is safer for "now").
- `transform_storage`: A variable (usually `tf::StampedTransform`) where the result will be written.

3. Example Code Snippet

Imagine you have a point `(x, y)` detected by the laser. You want to save it into a global map.

C++

```
#include <tf/transform_listener.h>


// 1. Create the listener (do this once, e.g., in class constructor)
tf::TransformListener listener;


void laserCallback(const sensor_msgs::LaserScan::ConstPtr& msg) {
    // We want to transform from "laser_frame" to "map"
    tf::StampedTransform transform;


    try {
        // 2. Ask the listener for the math
        // "Give me the transform to move data FROM laser_link TO map,
        //  using the most recent data available."
        listener.waitForTransform("map", "laser_link", ros::Time(0), ros::Duration(1.0));
        listener.lookupTransform("map", "laser_link", ros::Time(0), transform);


        // 3. Use the result
        // Example: The laser found a point at (2.0, 0.0) relative to itself
        tf::Vector3 point_in_laser(2.0, 0.0, 0.0);


        // Apply the transform math
        tf::Vector3 point_in_map = transform * point_in_laser;
```

```
        ROS_INFO("Obstacle is at Map X: %f, Y: %f", point_in_map.x(), point_in_map.y());


    } catch (tf::TransformException &ex) {

        ROS_ERROR("%s", ex.what());

    }

}
```

Why `waitForTransform`?

This is a common exam trick. Since `tf` is sent over the network, there is a delay. If you ask for a transform at `ros::Time::now()`, the listener might not have received that data packet yet (it might arrive 5ms later).

- **Without waiting:** `lookupTransform` throws an exception (data missing).
- **With `waitForTransform`:** The code pauses (blocks) for a short time (e.g., 1.0s) to see if the relevant data arrives before failing.

Summary for the Exam

1. `tf` is a distributed system for tracking coordinate frames.
2. `TransformListener` listens to `/tf` and buffers data locally.
3. `lookupTransform` computes the translation/rotation matrix between two frames.
4. **Direction matters:** `lookupTransform("map", "laser", ...)` gives you the matrix Tmap←laser (transforms points *from* laser *to* map).